

How to use the Kreation console

Kreation watches UPI payments as they are authorised and stops the ones that are scams. This guide takes you from never having seen it to reading every screen, running your own data through it, and knowing what each number means. It assumes you know nothing about fraud systems.

Live at kreation-upi.vercel.app · no login · works on a phone

Second edition, September 2026. Adds the **Your data** screen, the single-payment composer, the durable audit record, and what changed on the console because of them.

What this is

In most fraud, someone steals your card or your password and spends your money. Banks are good at catching that, because you did not make the payment.

An **authorised push payment scam** is different, and it is the one this system exists for. Someone calls pretending to be the police, a bank officer, an employer or a relative, and talks the victim into paying them. The victim taps the buttons themselves — their PIN, their phone, their bank app. To the bank it looks like an ordinary transfer, so it goes through, and by the time anyone reports it the money has been moved through a chain of accounts and withdrawn.

Kreaton reads every payment at the moment it is authorised — in well under a millisecond — and decides whether it should go through, whether the payer should be asked to confirm, or whether it should be stopped. It does that by weighing twelve signals: how unusual the amount is for this payer, whether they are on a call right now, how old the receiving account is, how fast that account forwards money onward, and so on.

Everything shipped with the console is simulated. The payments, the people and the scams are generated from a synthetic dataset, so no real person's money or data appears anywhere. The figures are real measurements of the system, taken on that synthetic data. If you bring your own file (page 9), the console then measures on that instead — and tells you so on every screen.

The three outcomes

Every payment ends up in exactly one of three states. These three words, and the three shapes beside them, appear on every screen — learn them once and the rest of the console reads itself.

I Let through	396	O Held to verify	2	× Blocked	2
Matches how this payer normally pays. It settles untouched.		Something is off. The payer confirms who they are paying before it moves.		Almost certainly a scam. The money never leaves the account.	

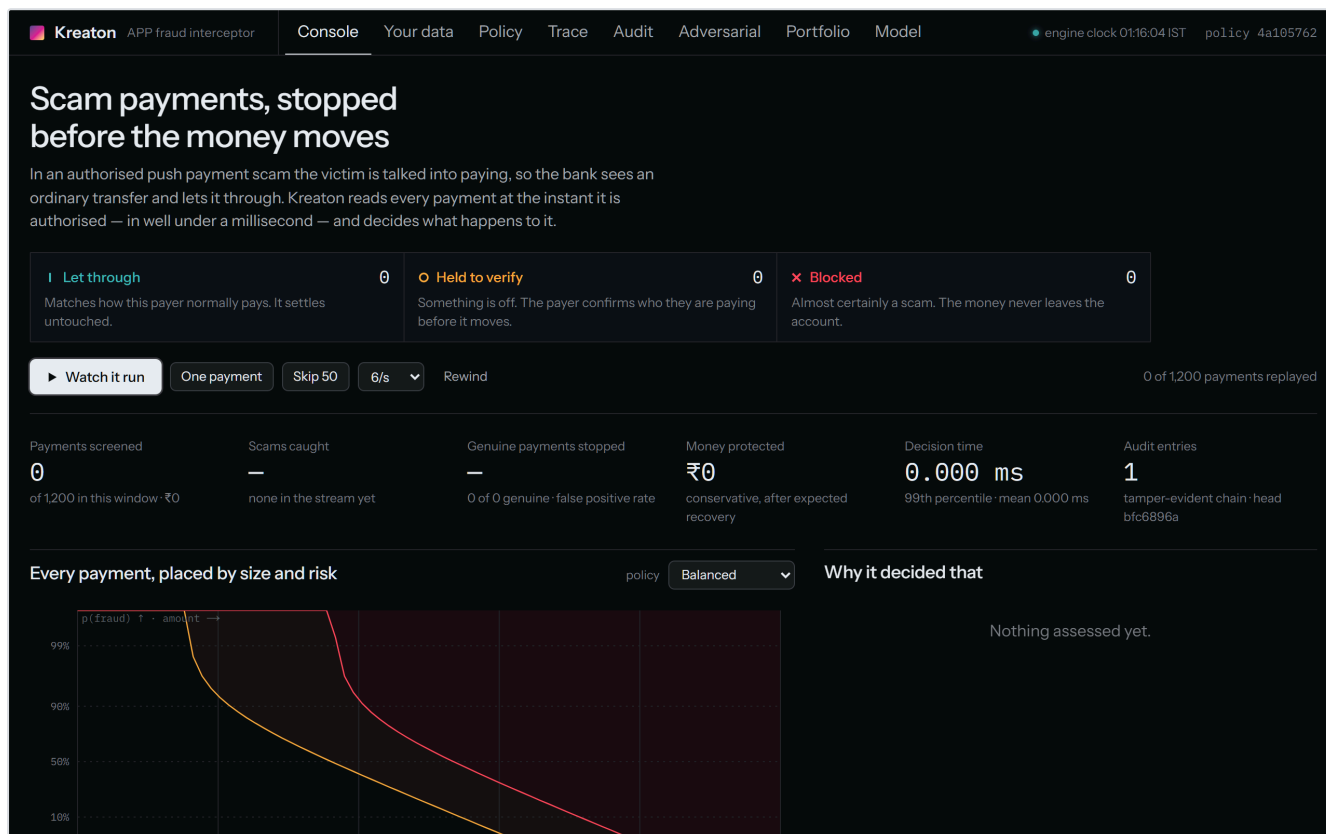
The key at the top of the console. The counts are live and match the ribbon and the feed below.

OUTCOME	WHAT IT MEANS	HOW OFTEN
Let through	Matches how this payer normally pays. It settles, untouched.	Almost every payment.
Held to verify	Something is off. The payer is asked to confirm who they are paying — in a way the fraudster on the phone cannot simply relay — before it moves.	A few in a thousand.
Blocked	Almost certainly a scam. The money never leaves the account.	A few in a thousand.

Your first five minutes

1. Open the console

Go to kreaton-upi.vercel.app. You land on the console with every counter at zero. Nothing is broken; nothing has been replayed yet.



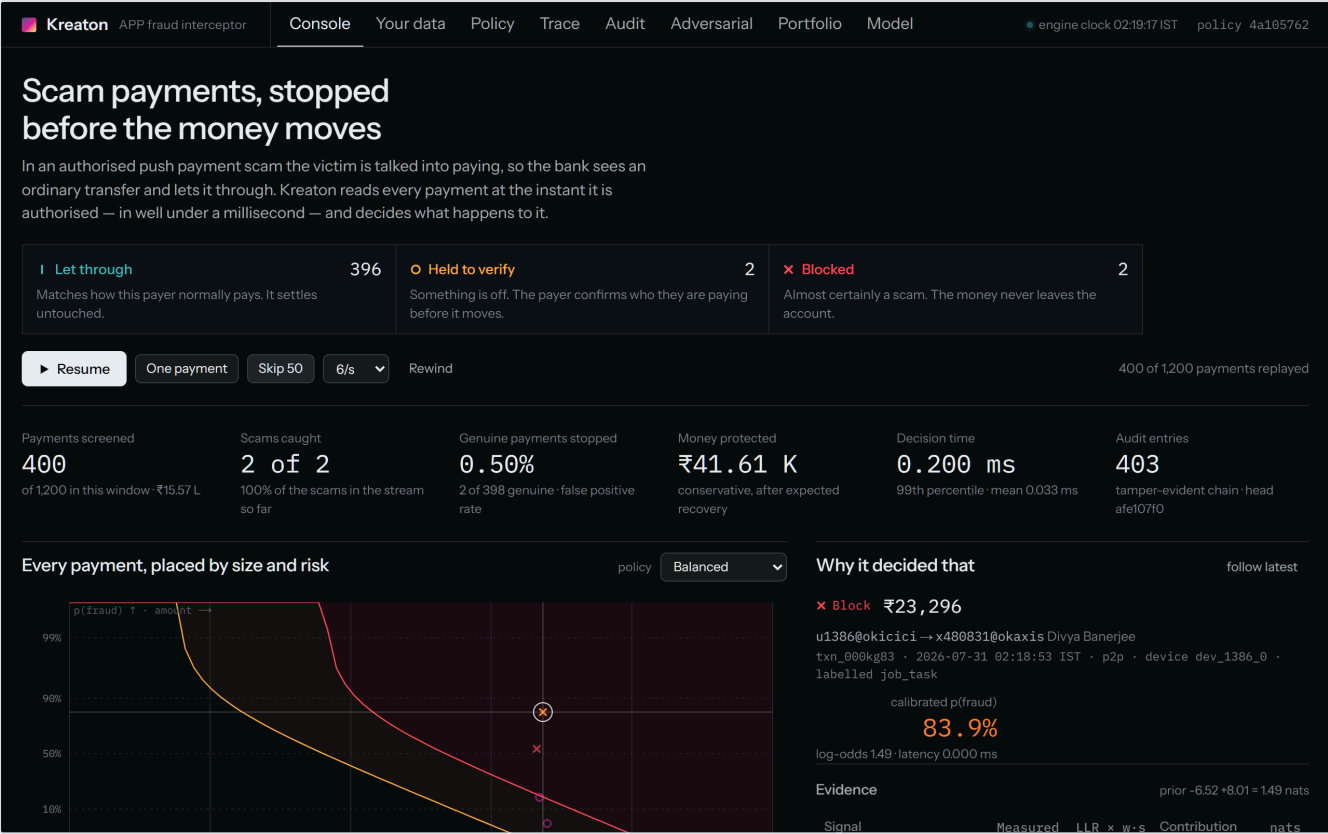
Fresh. The key reads 0 / 0 / 0 and the large button says **Watch it run**.

2. Press Watch it run

Payments start flowing through the engine at six per second. Each one is scored, decided, and written to the audit record as it passes. The clock in the top right is the engine's clock — it moves in payment time, not real time, so it runs faster than yours.

3. Let it run for about thirty seconds

You will see the **Let through** counter race ahead while **Held** and **Blocked** barely move. That imbalance is the whole problem in one picture: scams are rare, and a system that intervened too eagerly would block thousands of real customers to catch a handful of scams.



Mid-replay, 400 payments in. Counters at the top, measured statistics below them, the ribbon chart and the assessment panel below that.

4. Read the six statistics



FIGURE	WHAT IT TELLS YOU
Payments screened	How many have gone through the engine, and their total value.
Scams caught	Of the scams in the stream so far, how many were held or blocked. Shown only when the data has ground truth.
Genuine payments stopped	The false-positive rate. Every one is a real customer inconvenienced, so lower is better.
Money protected	Money that would have been lost had the payment gone through, counted conservatively after assuming some would have been recovered anyway.
Decision time	How long one decision takes, at the 99th percentile — the slow ones, not the average.
Audit entries	Records in the tamper-evident chain, and the fingerprint of the latest one.

5. Use the feed filter

Below the chart, the feed lists every payment. Since almost all of them are let through, press **Only the ones it stopped** to see just the interventions. This is the fastest way to see what the system is for.

Time	Paid by	Paid to	Amount	Scam risk p(fraud)	Decision	Why leading signal
02:18:53	u1386@okicici · known scam	x480831@okaxis Divya Banerjee	₹23,296	■ 83.9%	✗ Block	APP-AMT-6 · ₹23,296 is +4.2 robust deviations from this payer median payment of ₹768 across 35 transactions.
01:45:56	u1386@okicici · known scam	x480831@okaxis Divya Banerjee	₹21,068	■ 54.7%	✗ Block	APP-AMT-6 · ₹21,068 is +4.1 robust deviations from this payer median payment of ₹755 across 34 transactions.
01:41:57	u1110@okicici	m197@okicici Mehta Mobiles	₹22,050	■ 15.0%	○ Hold	APP-AMT-6 · ₹22,050 is +4.3 robust deviations from this payer median payment of ₹390 across 178 transactions.
01:40:01	u1455@oksbi	m484@okaxis Reddy Kirana	₹24,999	■ 5.89%	○ Hold	APP-AMT-7 · ₹24,999 is +5.9 robust deviations from this payer median payment of ₹89 across 378 transactions.

The same feed, filtered. Each row shows the amount, the scam risk, the decision, and the leading reason in plain words — with the reason code beside it for anyone who needs the precise name.

6. Click any row

The panel on the right fills in with why that payment got that decision. On a wide screen it stays beside the feed as you scroll; on a narrow one it comes to you.



A real blocked payment. Every signal is listed with what it measured, how much it moved the verdict, and in which direction. The contributions add up exactly to the total shown against **prior** at the top of the table — that is what makes every decision reconstructible by hand. Signals marked **quiet** did not fire; their small negative contributions are the engine noting the absence of a warning sign, and they are part of the reasoning too.

Expected cost of each action

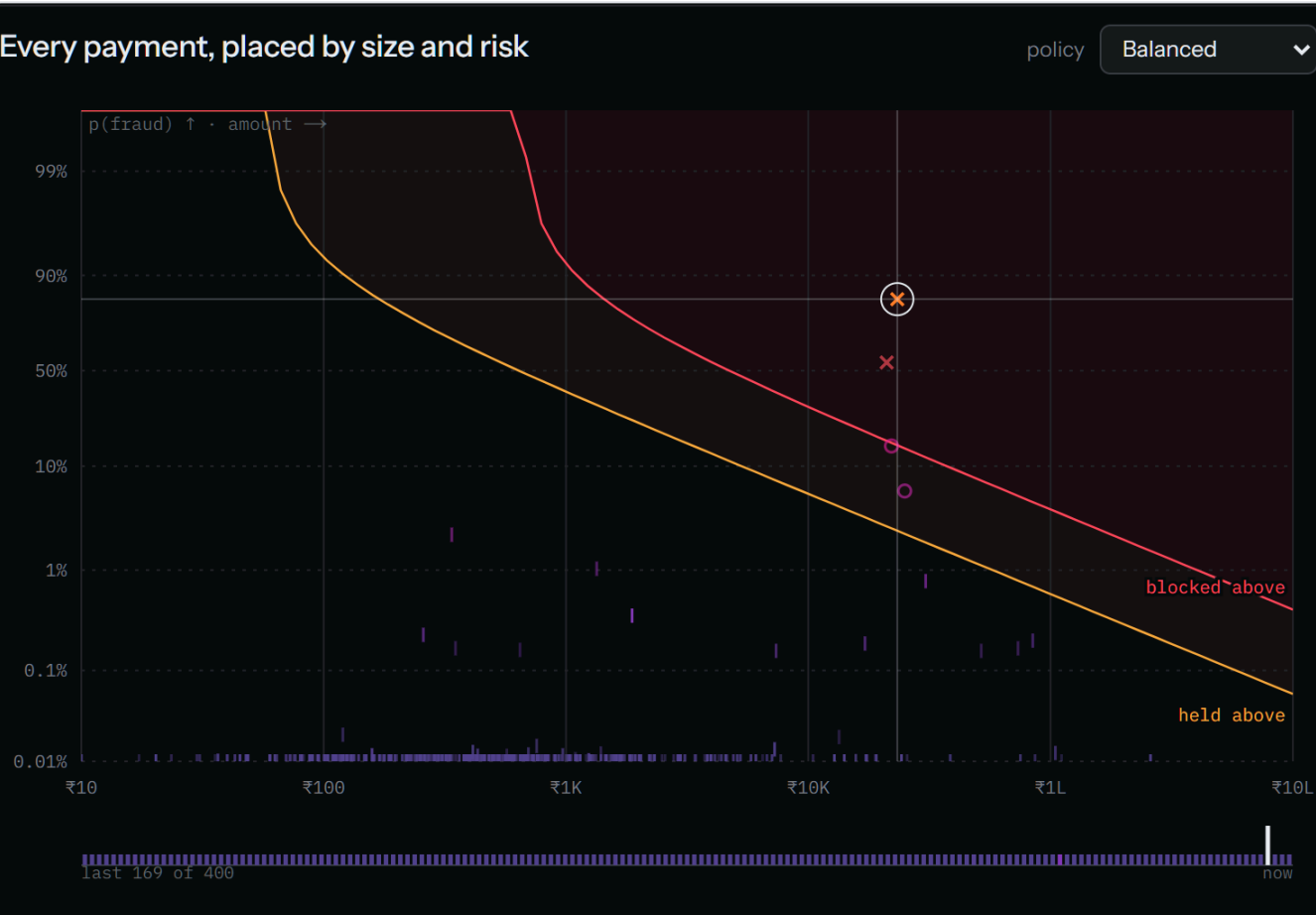
margin ₹6,018 over runner-up

Action	Liability	Friction	Total
I Approve	₹18,326	₹0	₹18,326
O Hold	₹6,414	₹86	₹6,500
X Block cheapest	₹0	₹482	₹482

Liability = $p \times \text{amount} \times \text{unrecovered share} \times \text{liability share}$. Recovery at the report lag of 240 min is 6.21%.

Below the evidence, **Expected cost of each action**. What approving, holding and blocking would each have been expected to cost, in rupees, given the probability and the amount. The cheapest is taken. There is no threshold anybody picked; this arithmetic is the decision, and it is the part that makes the system defensible to a regulator.

The chart



Every payment is a mark. Left to right is the amount, from ₹10 to ₹10 lakh on a log scale. Bottom to top is how likely the engine thinks it is to be a scam. Almost everything sits in the bottom band: ordinary payments, correctly judged harmless.

The two curved lines are the **decision boundaries**. Above the lower line a payment is held; above the upper line, blocked. They slope down to the right because the boundary moves with

the amount: the cost of wrongly approving a ₹5 lakh payment is far higher than wrongly approving ₹500, while the cost of wrongly holding either is about the same. The lines are not a threshold somebody picked — they fall out of the cost arithmetic, and they move when you change the policy in the dropdown.

Try a scam on it yourself

Scams are rare in the stream, so a live demonstration would wait a long time to see one arrive on its own. The injector builds one on demand against a real payer from the feed, with the context fingerprint that scam actually leaves, and pushes it through the same path as everything else.

Try a scam on it yourself

runs a known scam pattern against a payer from the feed, at the engine clock

Digital arrest

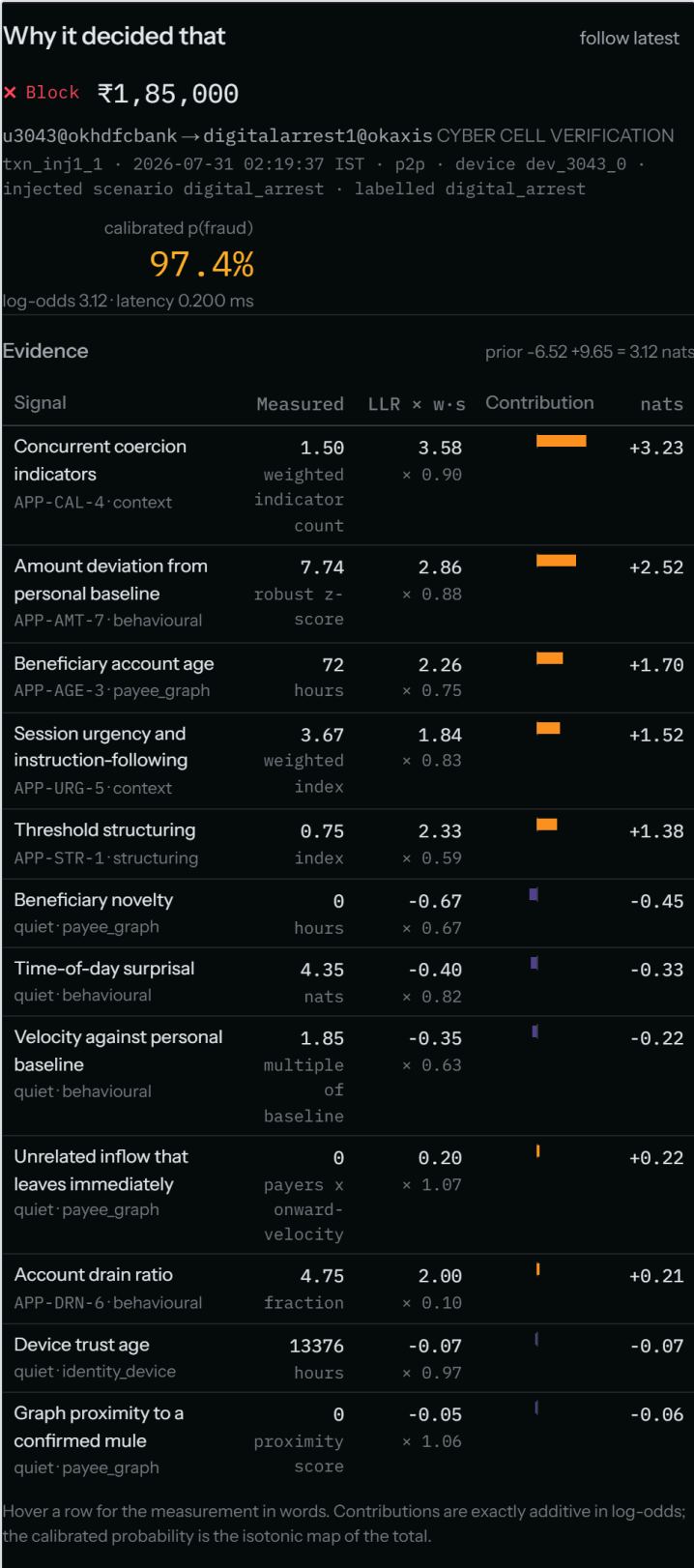
Inject

Victim held on a video call by someone posing as police, told to transfer savings to a "verification" account.

Expected: Call concurrency, urgency, novelty and amount deviation all fire.
Expected-cost holds or blocks; an on-device passcode is disqualified because the coach is on the line.

Pick a scenario, read what it is and what should happen, press **Inject**. Injected payments are marked as such and are excluded from the measured statistics, so the numbers on screen stay honest.

SCENARIO	WHAT IT IS	WHAT SHOULD HAPPEN
Digital arrest	A victim held on a video call by someone posing as police, told to transfer savings to a "verification" account.	Blocked. Call, urgency, novelty and amount all fire.
KYC update	An urgent message that the account will be frozen unless a fee is paid to a pasted identifier.	Blocked.
Structuring	An extraction split into payments each below a monitored limit.	The first passes; the window then crosses the threshold and the next is blocked.
Split re-attempt	A held payment retried in two halves to a fresh account.	Held, then linked to the open hold, then escalated.
Context suppression	The same extraction with every attacker-controllable indicator turned off: no call, typed identifier, unhurried.	Usually passes under Balanced; held under Liability-first. Try this one if you only try one.
Legitimate large payment	A big payment to a beneficiary the payer has paid for months.	Let through.



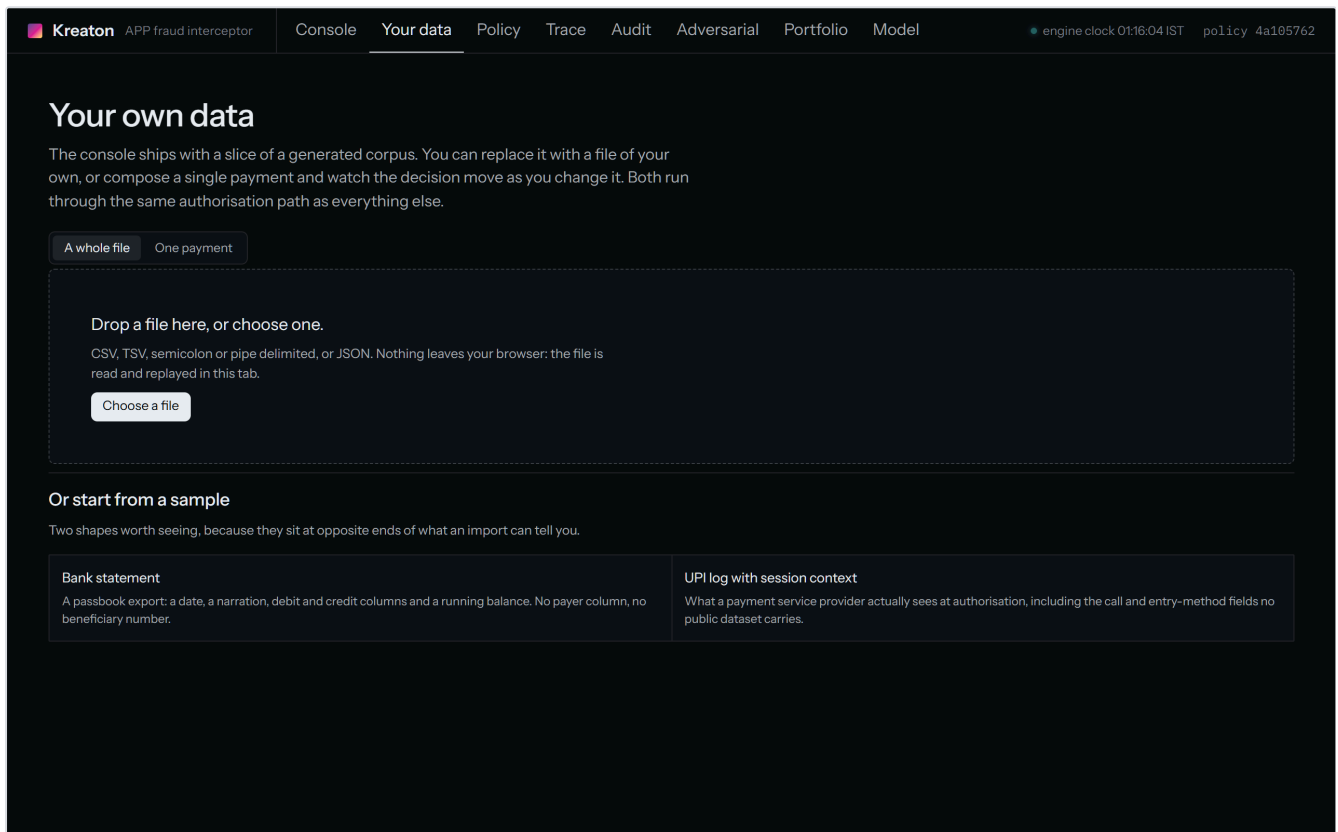
The digital-arrest scenario, injected and blocked. The header line says **injected scenario** so nobody mistakes it for something the stream produced. Compare the evidence with the previous panel: here the coercion indicators, urgency and beneficiary novelty all fire, which is the fingerprint this scam leaves.

Try **"Context suppression" if you only try one**. It is the case the system is worst at, and it is in the menu on purpose. A fraud tool that only demonstrates its successes is not telling you what it is worth. Run it on Balanced, then change the policy dropdown to Liability-first and inject it again.

Your data new in this edition

The console ships with a slice of the generated corpus. The **Your data** screen lets you replace it with a file of your own, or compose a single payment by hand and watch the decision move as you change it. Both run through exactly the same engine as everything else — nothing on this screen is a simulation of the product.

Replaying a file



Drop a file, choose one, or start from a sample. CSV, TSV, semicolon- or pipe-delimited text, or JSON. Nothing leaves your browser: the file is read and replayed in this tab.

Two samples are built in because they sit at opposite ends of what an import can tell you. The **bank statement** is what most people actually have: a date, a narration, debit and credit columns, a running balance — and no payer column, no beneficiary number, no session context. The **UPI log** is what a payment provider sees at authorisation, including the call and entry-method fields no public dataset carries.

Step 1 — confirm the mapping

The importer reads your column headings and proposes which engine field each one feeds. You confirm or change every line before anything is scored. This matters: bind the wrong column to the payer and every behavioural baseline is rebuilt against the wrong identity, and no number downstream would look wrong.

Kreaton
APP fraud interceptor
Console
Your data
Policy
Trace
Audit
Adversarial
Portfolio
Model
engine clock 01:16:04 IST
policy 4a105762

Your own data

The console ships with a slice of a generated corpus. You can replace it with a file of your own, or compose a single payment and watch the decision move as you change it. Both run through the same authorisation path as everything else.

A whole file
One payment

Bank statement (sample)

5 columns - delimited with ","

Choose a different file

Layout

Bank statement

A passbook or account statement: a date, a narration, and separate debit and credit columns.

Which column is which

Anything left unsupplied gets the substitute named beside it. Substitutes are not neutral — the ones marked ● *quiet* hand the engine the reading an attacker would want.

WHO PAID WHOM

Payment reference Unique reference for the payment. Used as the key everywhere in the console.	Not supplied	Numbered from the row position, imp_000001 upward.
Payer Who is paying. Every behavioural signal is computed against this identity, so it must be stable across rows.	Not supplied	
Payer UPI ID The payer UPI handle, when the dataset has one.	Not supplied	Derived from the payer identifier as <payer>@imported.
Beneficiary Who is being paid. The payee-graph signals key on this, so a dataset that never repeats a beneficiary loses that whole group.	Not supplied	

Replay this file
The engine starts from nothing: no prior profiles, no prior intelligence. The first 50,000 payments are replayed.

A bank statement, recognised as such. The layout dropdown offers presets for shapes it knows — a Kreaton export, PaySim, a statement — and "matched from the column names" for everything else.

WHO PAID WHOM

Payment reference Unique reference for the payment. Used as the key everywhere in the console.	Not supplied	Numbered from the row position, imp_000001 upward.
Payer Who is paying. Every behavioural signal is computed against this identity, so it must be stable across rows.	Not supplied	
Payer UPI ID The payer UPI handle, when the dataset has one.	Not supplied	Derived from the payer identifier as <payer>@imported.
Beneficiary Who is being paid. The payee-graph signals key on this, so a dataset that never repeats a beneficiary loses that whole group.	Not supplied	
Beneficiary UPI ID The beneficiary UPI handle, when the dataset has one.	Not supplied	Derived from the beneficiary identifier as <payee>@imported.
Beneficiary name The name of the beneficiary at the receiving institution. Shown throughout the console and in the audit.	Narration	

Replay this file
The engine starts from nothing: no prior profiles, no prior intelligence. The first 50,000 payments are replayed.

Each engine field, what it is for, which column feeds it, and — when nothing does — what the engine will assume instead. Fields marked with an amber dot are left at the value an **attacker would choose** when unmapped; the cost of a missing column is visible in the mapping itself, not only in the report below.

Two things a statement lacks are handled rather than refused. With **no payer column**, the whole file is read as one account holder — which is what a statement is. With **no beneficiary number**, the narration folded to a stable form becomes the identity, so "RAMESH TRADERS", "Ramesh Traders" and "UPI/RAMESH TRADERS." are one beneficiary rather than three. That is what lets the receiving-side signals work on a statement at all. Only the amount has no substitute.

Step 2 — check how the first rows read

How the first rows read

after the mapping above, before anything is scored

Reference	When	Payer	Beneficiary	Amount	Context
imp_000001	2026-04-02 05:30:00 IST	account_holder	BIG BAZAAR RETAIL	₹2,340	typed
imp_000002	2026-04-03 05:30:00 IST	account_holder	RAMESH TRADERS	₹1,250	typed
imp_000003	2026-04-05 05:30:00 IST	account_holder	Ramesh Traders	₹2,000	typed
imp_000004	2026-04-08 05:30:00 IST	account_holder	ELECTRICITY BOARD	₹3,180	typed
imp_000005	2026-04-11 05:30:00 IST	account_holder	RAMESH TRADERS.	₹1,700	typed
imp_000006	2026-04-14 05:30:00 IST	account_holder	SWIGGY ORDER	₹640	typed
imp_000007	2026-04-18 05:30:00 IST	account_holder	CYBER CELL VERIFICATION	₹89,000	typed

The first rows after the mapping, before anything is scored. Credits have already been set aside — a salary is money in, not a payment.

Step 3 — read what the file cannot show

This is the most important part of the screen, and it cannot be skipped.

What this file can and cannot show

from the rows read so far

Payments	Payers	Beneficiaries	Ground truth
7	1	5	none
of 8 rows	median 7 payments each	29% repeat	accuracy cannot be scored

This file carries no session context, so the engine sees every payment as if the attacker had suppressed every indicator they control: no call, no urgency, a typed identifier, a trusted device. That is the hardest position for the model and the weakest one in the adversarial report. Detection here is a floor, not a representative figure.

No beneficiary onward-velocity column, so the fan-in signal cannot separate a mule from a busy merchant. That signal is the most specific mule indicator the engine has.

Payers have a median of 7 payments each. Behavioural baselines need history; with this little, amount deviation and time-of-day surprisal are being measured against almost nothing.

No ground-truth column, so the run reports what the engine decided but cannot report whether it was right.

Assumed: No payer column, so every row is read as the same account holder. Behavioural baselines are built for that one payer. No beneficiary identifier column, so beneficiaries are identified by their name.

12 fields left at the value an attacker would choose

8 fields filled in harmlessly

Replay this file The engine starts from nothing: no prior profiles, no prior intelligence. The first 50,000 payments are replayed.

What this file can and cannot show. Rows kept and dropped by reason, payers and how much history each has, how often beneficiaries repeat, whether there is ground truth — and the cautions in plain language.

Almost no real file carries UPI session context. A bank statement does not know whether the payer was on a call, whether the identifier was pasted, whether the device was new. When those columns are absent the engine does not treat them as neutral — it sees every payment as though the attacker had suppressed every indicator they control. That is the hardest case for the model and the weakest one in the adversarial report. A detection rate measured on such a file is a **floor**, not a representative figure, and the console says so beside the number rather than letting it stand alone.

Step 4 — replay it

Loaded

7 payments from Bank statement (sample) are now the corpus this console replays. The feed, the assessment panel, the ledger and the policy studio all read from it.

Watch it run

Import another file

Go back to the shipped corpus

Loaded. The feed, the assessment panel, the ledger and the policy studio all now read from this file.

KreatonAPP fraud interceptor

ConsoleYour dataPolicyTraceAuditAdversarialPortfolioModel

engine clock 05:30:00 ISTpolicy 4a185762

Replaying Bank statement (sample), 7 payments. Everything below is measured on that file, which carries no ground truth, so the scams-caught and false-positive figures cannot be filled in.

What this file can and cannot show

Scam payments, stopped before the money moves

In an authorised push payment scam the victim is talked into paying, so the bank sees an ordinary transfer and lets it through. Kreaton reads every payment at the instant it is authorised — in well under a millisecond — and decides what happens to it.

I Let through 6	O Held to verify 0	X Blocked 1
Matches how this payer normally pays. It settles untouched.	Something is off. The payer confirms who they are paying before it moves.	Almost certainly a scam. The money never leaves the account.

▶ Resume

One payment

Skip 50

6/s

Rewind

7 of 7 payments replayed

Payments screened7of 7 in this window · ₹1 L

Scams caught—none in the stream yet

Genuine payments stopped14.29%1 of 7 genuine · false positive rate

Money protected₹0conservative, after expected recovery

Decision time0.200 ms99th percentile · mean 0.071 ms

Audit entries8tamper-evident chain · head d12c01f6

Every payment, placed by size and riskpolicyBalanced

Why it decided thatfollow latest

p(fraud) ↑ · amount →

X Block ₹89,000

account_holder@imported → cyber_cell_verification@imported CYBER CELL VERIFICATION

imp_000007 · 2026-04-18 05:30:00 IST · p2p · device

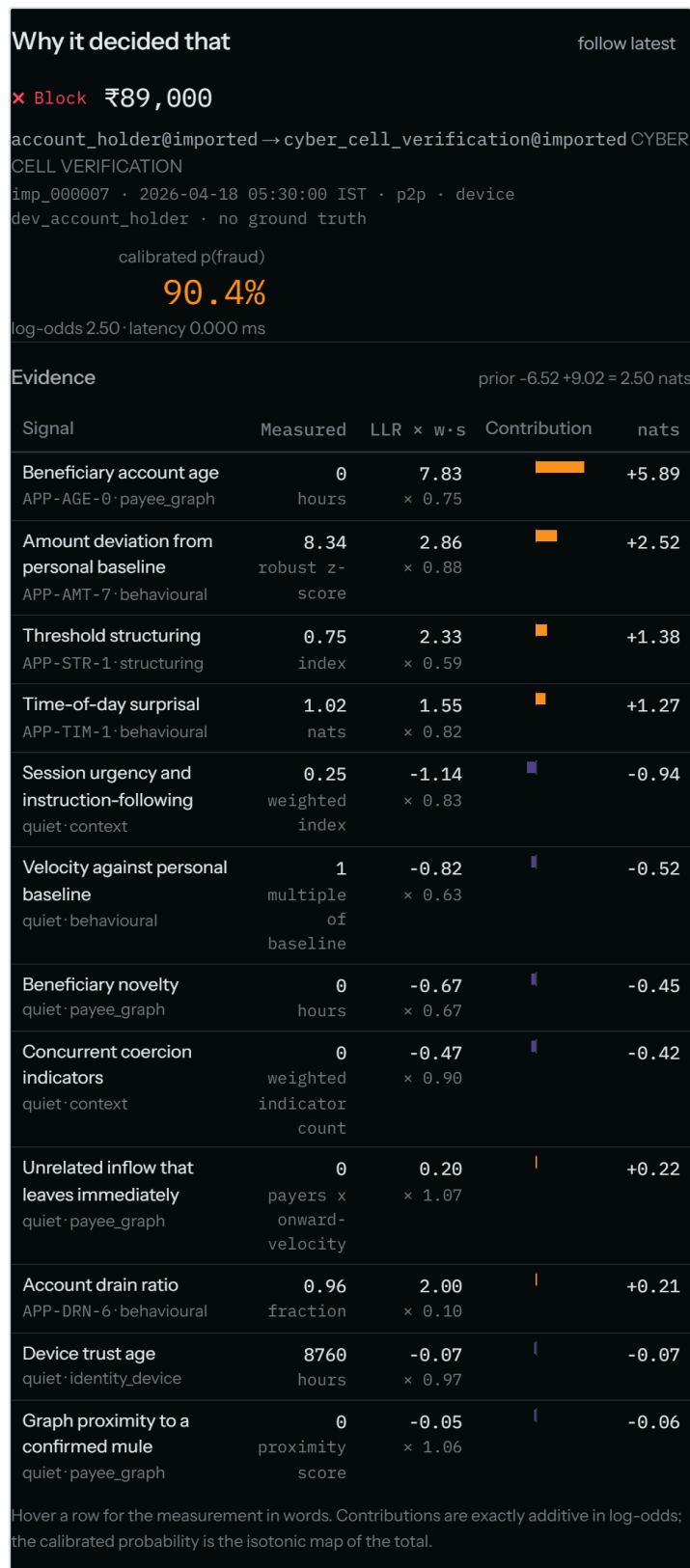
dev_account_holder · no ground truth

calibrated p(fraud)

The console with the sample statement loaded. The banner at the top says so, and says that this file carries no ground truth, so the scams-caught figure cannot be filled in. The ₹89,000 payment to "CYBER CELL VERIFICATION" was blocked at 90.4% — with no session context at all, from the amount, the new beneficiary, and the balance column showing it would have drained the account.

Kreaton · User manual

14 / 28



The assessment for that payment. Note **no ground truth** in the header line rather than "labelled legitimate": a file that never said anything about a payment has not called it legitimate, and the console does not put words in its mouth. Every session signal reads **quiet**, because a statement carries none of them — and the payment was still blocked on the receiving account's age, the amount against this payer's history, and the drain ratio, which used the statement's own balance column rather than an estimate.

To go back to the shipped corpus, press **Back to the shipped corpus** on either screen. Your imported file is kept in the browser, so a refresh no longer costs you the file you just mapped

— see "Things that will confuse you".

Composing one payment

The importer answers "how does the engine behave on my data". The second tab answers the other question people ask at a demonstration: "what happens if I change this one thing?"

Start from			
An ordinary payment A shop the payer uses often, scanned in person, no call.	Digital arrest Coached on a long call, identifier pasted from chat, savings-sized amount to an account opened days ago.	A careful attacker The same extraction with every indicator the attacker controls turned off. This is the hard case the evidence cap exists for.	A large genuine payment A builder the payer has paid for months. Big, but nothing else about it is unusual.

Four starting points, from an ordinary shop payment to a careful attacker who has turned off every indicator they control.

What was happening at the time

☒ On a call

Call length, minutes

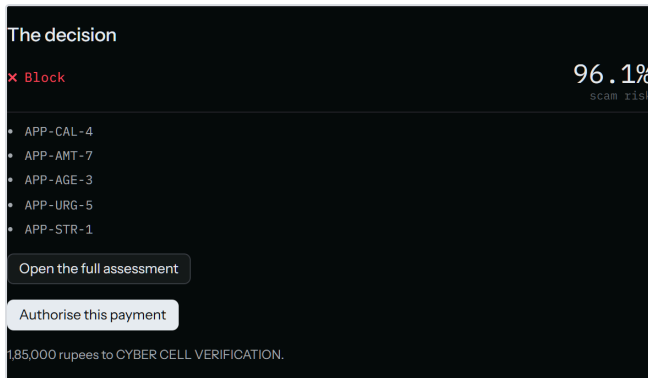
☐ Sharing their screen

☐ Remote access app running

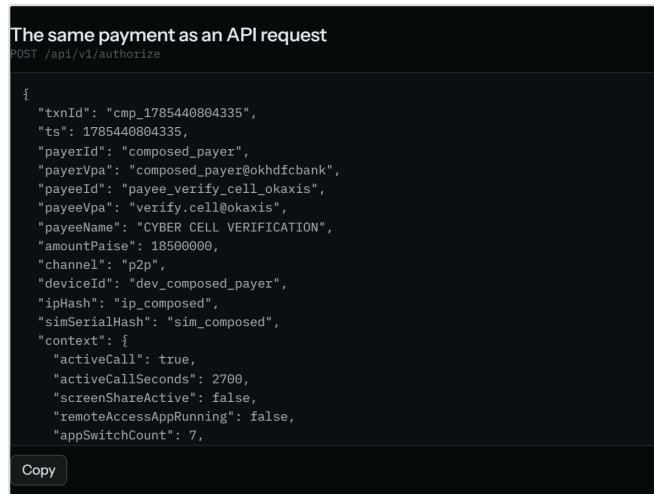
App switches before paying

Seconds from opening the app to paying

Every field the engine reads, as a control. Put the payer on a call, make it forty minutes, paste the identifier instead of choosing a contact, and watch the decision move.



The decision, the scam risk, and the leading reason codes.
Open the full assessment takes you to the complete evidence table.



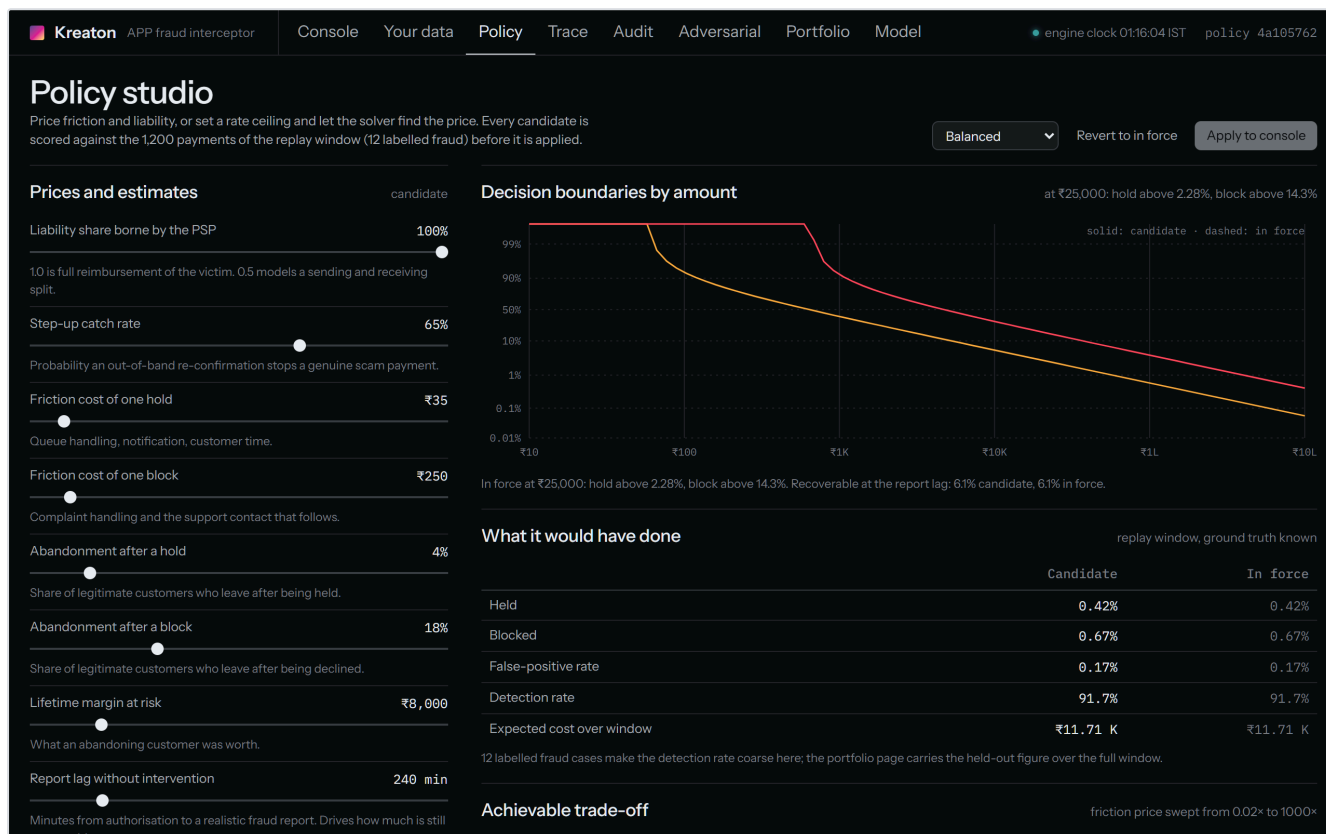
The same payment as the request body for `POST /api/v1/authorize`. The field you just changed on screen is the field an integrator would have to populate.

Composed payments run through the real authorisation path, so they land in the feed and in the audit ledger like any other payment. Choose a payer from the feed to score against a real history; "someone with no history" is a legitimate thing to test, but amount deviation and time-of-day surprisal then have almost nothing to measure against.

The other six pages

The links across the top lead to six more screens. Each answers one question.

Policy — "what if we tuned it differently?"



The trade-off between stopping genuine customers and paying compensation, as prices a risk officer can move and as rate ceilings an operations team can set. Every candidate is priced against the scored population before it is applied, and applying it is written to the ledger.

Trace — "where would the money have gone?"

 **Kreaton** APP fraud interceptor

ConsoleYour dataPolicyTraceAuditAdversarialPortfolioModel

engine clock 01:16:04 ISTpolicy 4a105762

Mule chain trace

Time-decay recoverability for one payment: how much of it a freeze order can still reach, minute by minute, and the collection network it entered.

Recent payments worth tracing

intercepted, or scored above 1%

Nothing yet. Play the console or inject an episode, then come back.

For a stopped payment, the chain of accounts the money would have passed through and how recoverable it would have been at each point. Recoverability falls steeply with time: this is why stopping a payment beats chasing it.

Audit — "prove none of this was altered afterwards"

Kreaton

APP fraud interceptor

Console

Your data

Policy

Trace

Audit

Adversarial

Portfolio

Model

engine clock 01:16:04 IST

policy 4a185762

Compliance trail

Append-only, hash-chained. Each record commits to the digest of the one before it, so altering any entry invalidates every later digest and the verifier reports where.

Verify chain

Tamper a copy

Export JSONL

records sealed

1

head digest

b8feabb6f97646ed

commits to the whole history

verification

not run

recomputes every digest

tamper check

not run

alters one record in a copy

Query

filters are conjunctive

Transaction id

txn_...

Record kind

any

Decision

any

Reason code

APP-CALL-3

Minimum p(fraud), percent

1

1 record match

Records

newest first, up to 200

Seq	Time	Kind	Summary	Digest
0	09:49:32	MODEL_LOADED	10.0-kreaton-2026-4000x60 096614f59c00 fitted 2026-09-11-2026-09-12 09:49:32 IST	b8feabb6f976

Every decision is written to an append-only record where each entry carries a fingerprint of the one before it. Change any entry and every later fingerprint stops matching. The regulator's view: what was decided, when, under which model and which policy, with the evidence sealed in.

Kreaton · User manual

20 / 28

Adversarial — "what happens when the fraudster adapts?"

KreatonAPP fraud interceptor

ConsoleYour dataPolicyTraceAuditAdversarialPortfolioModel

engine clock 01:16:04 ISTpolicy 4a105762

Adversarial evaluation

6 attacks, 400 episodes each, launched against three defences tuned to the same friction budget of 0.15% of legitimate payments (measured on 190,223 held-out legitimate payments). An episode counts as caught when any payment in it is held or blocked. Model 1.0.0-kreaton-2026-4000x60, policy balanced.

Tuned rule

13.1%

315 of 2400 episodes · leaked ₹17.48 Cr

Fused score, single threshold

79.2%

1901 of 2400 episodes · leaked ₹8 Cr

Fused score, expected-cost boundary

73.8%

1772 of 2400 episodes · leaked ₹5.24 Cr

weakest position

25.0%

Context suppression

Defences under test

calibrated to a common friction budget

Defence	What it is	Measured FPR
Tuned rule	Intervene above a fixed amount, or above a lower amount when the beneficiary is new. Thresholds fitted to the same false-positive rate as the other systems. This is the shape of most controls actually in production. Fitted: intervene above ₹2.5 L, or above ₹1 L for a beneficiary under 24 h old.	0.14%
Fused score, single threshold	The same twelve-signal fused and calibrated score, compared against one global probability threshold. Isolates what the score contributes from what the decision layer contributes. Fitted: intervene at $p \geq 0.0142$.	0.15%
Fused score, expected-cost boundary	The full system. The same score, but the intervention boundary is derived per payment from expected compensation liability against expected friction, so it moves with the amount.	0.15%

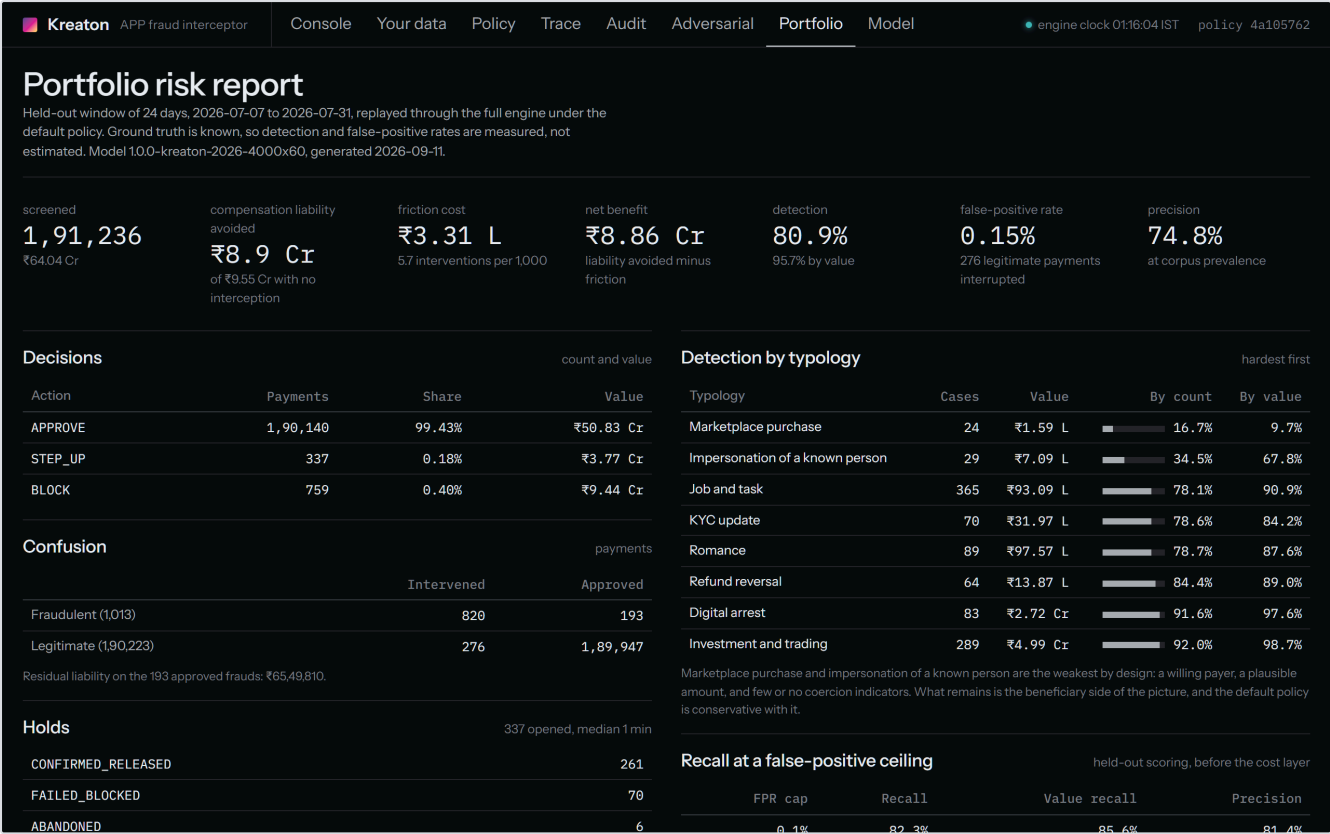
Episodes caught, by attack

95% Wilson intervals

Attack	Tuned rule	Fused score, single threshold	Fused score, expected-cost boundary	Gain over rule	Ablation
Threshold structuring	0.0% 0%–1% · ₹5.81 Cr	98.8% 97%–99% · ₹5.17 L	99.0% 97%–100% · ₹4.69 L	+99.0 pp intervals disjoint	+0.5 pp not significant
False baseline building	11.3% 9%–15% · ₹3.13 Cr	81.5% 77%–85% · ₹3.51 Cr	65.3% 60%–70% · ₹2.37 Cr	+54.0 pp intervals disjoint	+6.8 pp not significant
Delayed extraction	11.8% 9%–15% · ₹1.55 Cr	81.5% 77%–85% · ₹14.06 L	63.2% 58%–68% · ₹22.24 L	+51.5 pp intervals disjoint	+7.0 pp not significant
Context suppression	10.8% 8%–14% · ₹1.59 Cr	15.5% 12%–19% · ₹3.25 Cr	25.0% 21%–29% · ₹1.64 Cr	+14.3 pp intervals disjoint	+0.0 pp not significant

Six attacks run against three defences, all tuned to interrupt the same number of genuine customers so the comparison is fair. It includes the attacks the system is worst at.

Portfolio — "how well does it actually do?"



Model — "what is inside it?"

Kreaton

APP fraud interceptor

Console

Your data

Policy

Trace

Audit

Adversarial

Portfolio

Model

engine clock 01:16:04 IST

policy 4a105762

Model card

1.00-kreaton-2026-4000x60, fitted 2026-09-11, digest a6a137e62a986ac5. Twelve weight-of-evidence signals fused additively in log-odds with group correlation shrinkage and an asymmetric cap on attacker-controllable evidence, then calibrated by isotonic regression. Fitted on 2,86,854 payments (421 fraudulent) and evaluated on the 1,91,236 that followed in time.

held-out ROC AUC

0.9960

PR AUC

0.8895

prevalence 0.30%

KS

0.949

recall at 1% FPR

93.4%

Brier

0.00179

calibration error

0.0019

worst bin 0.022

strongest single signal

0.946

Amount deviation from personal baseline

Signals

weights fitted by IRLS in 6 iterations; standalone AUC is a guard against the corpus, not a ranking

Signal	Group	Measures	Weight	Shrink	LLR range	Alone AUC
▶ Beneficiary novelty	payee_graph	Hours elapsed since this beneficiary was first added or first paid by this payer. (hours)	0.779	0.056	-3.31 to 3.10	0.935
▶ Amount deviation from personal baseline	behavioural	Robust z-score of the log amount against the payer own historical log-amount distribution. (robust z-score)	1.160	0.105	-5.01 to 2.86	0.946
▶ Time-of-day surprisal	behavioural	Negative log probability, in nats, of the transaction hour under the payer own hour-of-day histogram. (nats)	1.072	0.105	-0.40 to 1.55	0.520
▶ Device trust age	identity_device	Hours since the authorising device was first bound to this payer, forced to zero when the SIM changed inside the risk window. (hours)	0.972	0.000	-0.32 to 5.81	0.517
▶ Unrelated inflow that leaves immediately	payee_graph	Distinct payers to this beneficiary in the trailing 24 hours, multiplied by the fraction of inbound value moved onward within the hour. (payers x onward-velocity)	1.253	0.056	-5.31 to 1.36	0.785
▶ Beneficiary account age	payee_graph	Hours since this beneficiary was first observed anywhere in the network. (hours)	0.879	0.056	-0.86 to 7.83	0.668
▶ Concurrent coercion indicators	context	Weighted count of coercion indicators present at authorisation: call duration, screen sharing, and remote-access software. (weighted indicator count)	0.908	0.006	-1.54 to 9.88	0.682
▶ Session urgency and instruction-following	context	Composite of in-session application switches, how the payee	0.831	0.006	-1.43 to 1.84	0.815

The model card: what it was fitted on, every signal with its weight, and the held-out figures. Generated from the committed artefacts, not written by hand.

Kreaton · User manual

23 / 28

Glossary

Scam risk $p(\text{fraud})$	The engine's estimate that this payment is a scam, from under 0.001% to near 100%. Most payments sit far below 1%.
Genuine payments stopped false positive rate	The share of legitimate payments that got interrupted. Every one is a real customer inconvenienced, so lower is better.
Precision	Of the payments the system stopped, the share that really were scams. High precision means staff are not wading through false alarms.
Money protected liability avoided	Money that would have been lost and not recovered, had the payment gone through. Counted conservatively.
Mule account	An account that exists to receive scam money and pass it on quickly, often opened with a real person's stolen or rented identity.
Hops	How many accounts the money has been forwarded through. More hops means less chance of freezing it in time.
Step-up	The extra confirmation asked of a payer when a payment is held — chosen so the fraudster on the phone cannot simply relay it.
Contribution nats, log-odds	How much one signal moved the verdict. Positive pushes toward scam, negative toward legitimate. They add up exactly to the total.
Calibration	Whether the stated probabilities are honest. A well-calibrated 10% means it happens about one time in ten.
Decision time latency p_{99}	How long one decision takes. " p_{99} " means 99 out of 100 were at least this fast — fairer than the average, which hides the slow ones.
Ground truth	Whether the data says which payments really were scams. The shipped corpus has it throughout; your file may not, in which case accuracy is not reported at all.
Quiet value	What the engine assumes for a session field your file did not supply: no call, typed identifier, trusted device. Each is the reading an attacker would want.

Things that will confuse you

Reloading the page resets the shipped replay — but keeps your file

The engine runs inside your browser tab. A full reload (F5) sets the shipped replay back to zero. If you had imported a file, that file and how far you had replayed it are kept in the browser and restored automatically. Moving between pages using the links at the top keeps everything either way.

The clock is not real time

Playback is measured in payments per second, not minutes. The shipped window covers about four hours of traffic, so the clock in the top right jumps forward much faster than a real one. With an imported file the clock follows the file's own timestamps.

The dates are in 2026

The synthetic dataset was generated with a 2026 window. Nothing is wrong with your computer.

Scams you inject do not change the score

Injected and composed payments are marked as such and excluded from the measured statistics. If they counted, anyone could inflate the detection rate by injecting twenty obvious scams.

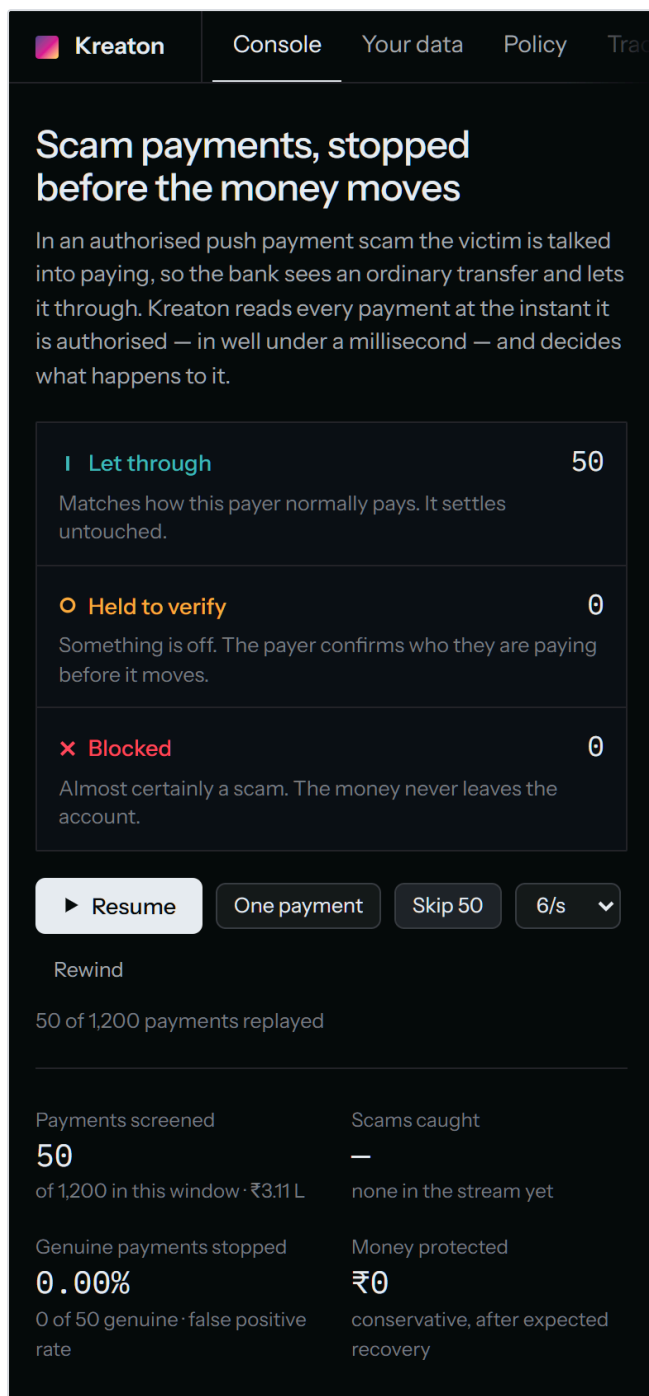
"Scams caught" says "—" on your file

Your file has no ground-truth column, so the console does not know which payments were scams and will not guess. You still get every decision and every reason; you do not get a score for them.

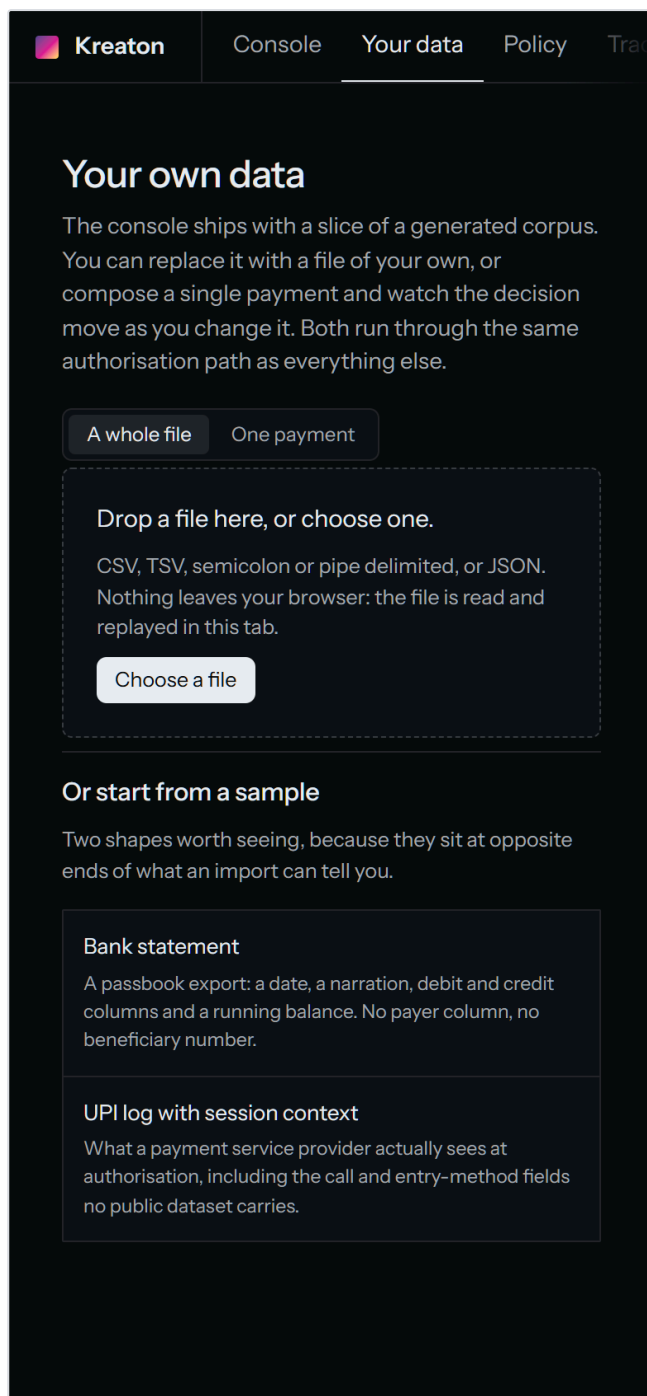
Nothing happens for a long time

That is the correct behaviour, not a broken page. Scams are a small fraction of payments. Use the feed filter or inject a scenario if you want to see interventions immediately.

On a phone



The console at 390px. The layout folds to one column and the feed becomes one card per payment.



Your data at 390px. The mapping rows stack; nothing scrolls sideways except tables inside their own frame.

For whoever maintains it

Where it lives

The site is hosted on Vercel at kreaton-upi.vercel.app, built from the `apps/web` folder of the repository. The code is on GitHub at [junekashif/Kreaton](https://github.com/junekashif/Kreaton); the submission copy is **Cognitia-IEM/KREATON**. Pushing to the main branch runs the checks and, if they pass, redeploys.

The API

GET <code>/api/v1/health</code>	model and policy digests, ledger head, persistence status
GET <code>/api/v1/authorize</code>	a complete example request body
POST <code>/api/v1/authorize</code>	authorise one payment
POST <code>/api/v1/authorize/batch</code>	authorise an ordered run, state carrying forward
POST <code>/api/v1/import</code>	match a file's columns and report; optionally replay it

```
pretty-print
{"ok":true,"model":{"version":"1.0.0-kreaton-2026-8000000","hash":"a0a137e2a908ac561d278a7a5cc025d6099550e21dc1a0b7887933a6fb1a68"},"policyHash":"4a105763f83ccdb6b8f0a64038cd3e9d405f3b5ed0b666561d5b349574626","ledger":{"records":1,"head":"0a4f346534b6ce9a233b30defb0e4941c37d088b8e92ec466c8090f747f88"},"persistence":{"backing":"google-sheets","buffered":1,"written":0,"failed":0,"dropped":0,"lastFlushAtMs":null,"lastError":"fetch failed (write ETIMEDOUT)}}
```

`/api/v1/health` on the live site. The **persistence** block says where decisions are being kept — **memory** when nothing durable is configured, **google-sheets** when the audit record is being copied to a spreadsheet, with counts of rows written, failed and dropped.

Where decisions are kept

The engine holds its working state in memory, because an authorisation has a budget of a few milliseconds. Durability sits behind that: after each decision has been returned, the sealed audit record, the payment and any hold are copied to a Google Sheet, in batches, so a judge or an auditor can open the record in a browser without being given a query tool. If the copy fails the decision is unaffected; the health endpoint shows **failed** and **dropped** counts and the last error. Setup — one service account, one shared spreadsheet, two environment variables — is in `docs/PERSISTENCE.md`.

The GitHub secrets

SECRET	WHAT IT IS	SENSITIVE?
<code>VERCEL_TOKEN</code>	Proves "this machine may deploy to our Vercel account".	Yes. Anyone holding it can deploy to or delete the account's projects.
<code>VERCEL_ORG_ID</code>	Which Vercel account the project belongs to.	Not really — an identifier, not a key.
<code>VERCEL_PROJECT_ID</code>	Which project within that account.	Not really.

They are stored as secrets rather than written into the code because everything in the repository is readable by anyone. Without them the deploy job skips itself and the site stays up as it was; you deploy by hand with `vercel deploy --prod` from the repository root. On Windows do not run `vercel build` first — it creates symbolic links, which Windows refuses without Developer Mode.

Built by Team Kreation for COGNITIA 2026, problem statement FINTECH-PS2. All payments, payers and scams shown are synthetic unless you supplied them. Screenshots in this guide were taken from the live console on 12 September 2026.